Multi Robot Planning and Coordination (16-891) Spring 2026 - The Robotics Institute

Homework 2 : Multi-Agent Path Finding (MAPF)

Instructor: Jiaoyang Li TA: Philip Huang, Jingtian Yan

Boxiang Fu

March 6, 2026

Collaborators:

1. F.D.C. Plato: Emotional support cat.

Task 1

Implemented in `kr_cbs.py`. Note that the `kr_cbs.py` implementation mentioned in the homework and code prototype uses point constraints and may not solve all test cases (e.g. `test_0.txt` with $k = 2$). A symmetric constraint implementation is implemented for Task 16/17 and provided in `kr_cbs_range.py` and can be ran by switching the solver to `--solver KRCBS-RANGE`.

Task 2

For $k \geq 1$, there is no need to find edge conflicts and impose edge constraints in kR-CBS. This is because for kR-CBS, an agent cannot visit a vertex within k timesteps of another agent visiting the same vertex. Letting $k = 1$, this exactly encodes an edge conflict. Therefore, detecting edge conflicts are redundant for $k \geq 1$. It is problematic to run kR-CBS with $k = 0$ exactly because of edge conflicts. Since letting $k = 1$ encodes edge conflicts, with $k = 0$ only vertex conflicts are encoded and not edge conflicts. This would fail to detect swaps and could return incorrect solutions where agents collide on edges. This is why an extra check was put in place for $k = 0$ in my implementation for `detect_first_collision_for_path_pair` under `kr_cbs.py`.

Task 3

Implemented in `ta_random.py`.

Task 4

Statistics from running 10 experiments are as follows:

Sum of costs:

1. 22
2. 20
3. 26
4. 32
5. 35
6. 35
7. 25
8. 28
9. 28
10. 26

Minimum: 20. Maximum: 35. Mean: 27.7. (Sample) Variance: 25.567. (Population) Variance: 23.01.

Task 5

Implemented in `ta_distance.py` and `hungarian.py`.

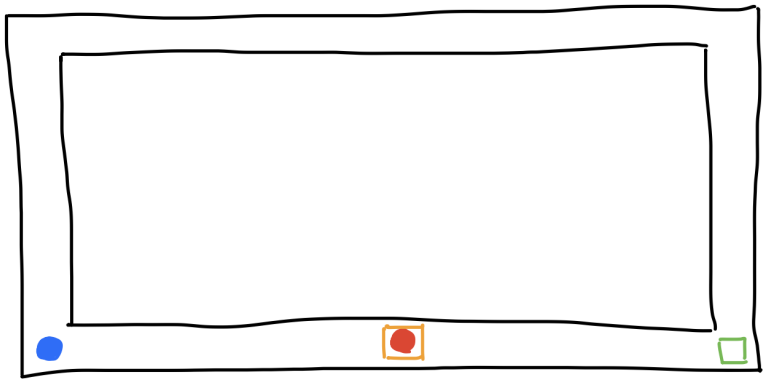
Task 6

With $k = 2$, the sum of costs is 16. This is lower than the mean and minimum from random assignment. This is expected since minimizing the distances is a much better metric for target assignment than naively assigning the tasks at random. Note however, it is still not optimal.

Task 7

The Figure below depicts a scenario where the Hungarian algorithm will yield suboptimal results. Note that in the Hungarian algorithm's implementation during tie-breaking (both theory and in `SciPy`), if an agent has 0 assignment cost (the agent is already at a goal), then any tie-breaking will favor this arrangement. This means that the Hungarian algorithm would allocate the red agent with the orange goal, and the blue agent with the green goal. This has an sub-optimal sum of

costs as the blue agent needs to travel the long route to reach the green goal since the red agent is already at its goal and is blocking the short route. The alternate assignment would have both agents going the short route, thereby having a lower (optimal) sum of costs.



Task 8

The Figure below depicts a scenario where the Hungarian algorithm will yield an invalid assignment. Note that in the Hungarian algorithm's implementation during tie-breaking (both theory and in `SciPy`), if an agent has 0 assignment cost (the agent is already at a goal), then any tie-breaking will favor this arrangement. This means that the Hungarian algorithm would allocate the red agent with the orange goal, and the blue agent with the green goal. This is an invalid assignment as the red agent is already at its goal and is blocking the corridor for the blue agent to reach to green goal. The alternate assignment would have both agents move to the right, thereby no blocking would occur and both agents would reach their goals.

Task 9

Implemented in `ta_cbs.py`.

Task 10

`test_1.txt` and $k = 2$:

TA-CBS:

- CPU time (s): 0.00

- Sum of costs: 16
- Expanded nodes: 3
- Generated nodes: 5

TA-Distance:

- CPU time (s): 0.00
- Sum of costs: 16
- Expanded nodes: 3
- Generated nodes: 5

Random:

- CPU time (s): 0.00
- Sum of costs: 24
- Expanded nodes: 2
- Generated nodes: 3

`test_2.txt` and $k = 2$:

TA-CBS:

- CPU time (s): 0.01
- Sum of costs: 20
- Expanded nodes: 4
- Generated nodes: 5

TA-Distance:

- CPU time (s): 0.00
- Sum of costs: 22
- Expanded nodes: 7
- Generated nodes: 11

Random:

- CPU time (s): 1.30
- Sum of costs: 49
- Expanded nodes: 1810
- Generated nodes: 3461

`test_3.txt` and $k = 2$:

TA-CBS:

- CPU time (s): 0.00
- Sum of costs: 9
- Expanded nodes: 1
- Generated nodes: 1

TA-Distance:

- CPU time (s): 0.00
- Sum of costs: 9
- Expanded nodes: 1
- Generated nodes: 1

Random:

- CPU time (s): 0.00
- Sum of costs: 22
- Expanded nodes: 4
- Generated nodes: 7

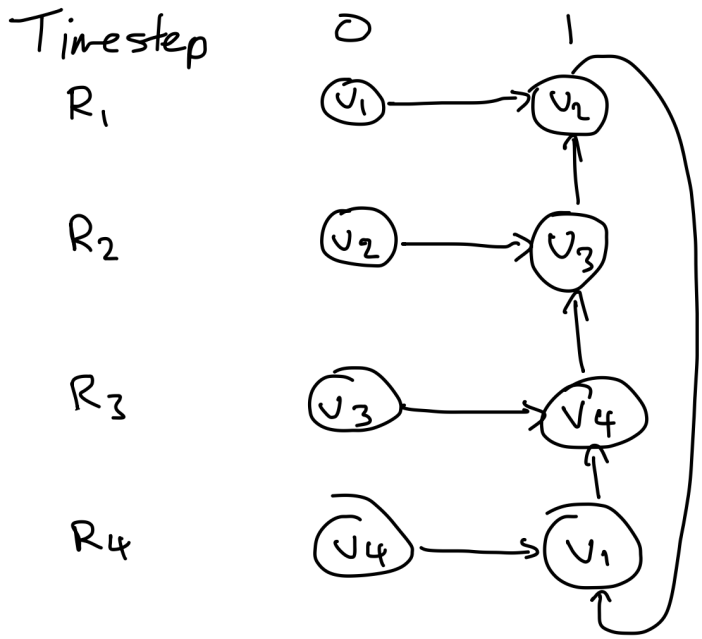
In terms of efficiency, both TA-CBS and TA-Distance are quite efficient in terms of CPU time. Although TA-Distance is slightly more efficient as it is more of an one-shot approach and does not need to maintain an open list. Its drawback is that it is not optimal (as seen in `test_2.txt`) and incomplete. Random performs significantly worse in CPU time on average due to very bad task allocations.

In terms of sum of costs, TA-CBS and TA-Distance are relatively cost efficient. TA-CBS represents an optimal lower bound for the sum of costs, so it is always the lowest out of the three. TA-Distance maintains a relatively low sum of costs, and occasionally achieves the optimal lower bound on some good heuristic allocations. Random's sum of costs is significantly higher due to random very bad task allocations.

Task 11

A greedy policy that commands agents to step forward along their paths as long as their next cell is unoccupied could lead to deadlocks and congestion. A deadlock could occur when each agent wants to go to the next vertex in a loop or narrow corridor. However, none can move under the greedy policy since each agent is blocking the subsequent agent. Additionally, it could also lead to congestion down a narrow corridor. Say an agent must wait for the opposing traffic to cross before entering the corridor. If it follows the greedy policy, it would enter the corridor prematurely and come face-to-face with opposing traffic. This will cause congestion and block the corridor.

Task 12



When robots use a TPG, each robot is only allowed to move to its next location if it has received messages from all incoming edges. In the example above, each robot wants to move into a vertex that is currently occupied by another robot. This dependency is circular (R1 must wait for R2 must wait for R3 must wait for R4 must wait for R1), so the incoming messages are never received as no robot would take the initiative to move (they are always waiting for its predecessor's message, but never receives one because of the circular dependency). This results in a deadlock and end up waiting forever.

Task 13

Implemented in `execution_manager.py`.

Task 14

Note: Reported time is "Average time" statistic as explained in Canvas announcements.

`test_0.txt` and $k = 0$:

Simulated average cost: 22.3s

`test_0.txt` and $k = 2$:

Simulated average cost: 20.8s (ran using `--solver KRCBS-RANGE`. Standard kR-CBS solver did not terminate)

`test_1.txt` and $k = 0$:

Simulated average cost: 11.6s

`test_1.txt` and $k = 2$:

Simulated average cost: 10.1s

On `test_0.txt`, $k = 2$ standard kR-CBS did not return a solution presumably due to single-timestep constraints implemented in Task 1 causes cascading re-collisions that cause the CBS search tree to explode. This was addressed by implementing symmetric constraint kR-CBS (which uses the solver `--solver KRCBS-RANGE`).

For both scenarios, we actually see that the average cost for $k = 2$ is lower than $k = 0$. This was something unexpected. A theoretical explanation is that the extra maintained distance perform better in realistic simulations because they avoid delay cascades due to the agents from being too close together for $k = 0$.

Task 15

The original simulator assumes an idealized discrete model where robots can start/stop/turn instantly and there is no physical dynamics. The SMART simulator takes all these realisms into account. This causes differences in robot behavior in the two simulations. In the original simulator, robots move exactly one grid cell per timestep; motion is perfectly synchronized; agents can accelerate, decelerate, and turn instantaneously; and paths are executed perfectly. In the SMART simulator, robots may take longer to move one grid cell per timestep; motion is not perfectly synchronized; agents need time to accelerate, decelerate, and turn; and paths may be subject to small deviations or delays. Because of this, the simulated execution time can differ from the idealized discrete cost.

Increasing the k parameter creates a greater temporal safety margin between agents. The increased spacing allows the physical dynamics to play out before causing a delay. In the original simulator, increasing k always causes the sum of cost to stay the same or increase. This is because it does not model physical dynamics, so no delays are caused by robots staying too close. Contrastingly, in the SMART simulator, increasing k could sometimes cause a decrease in execution time. This is because the additional spacing prevents robots from clustering too closely and reduces congestion. Increasing k by too much would eventually lead to increased execution time due to the paths being overly conservative.

Task 16

Note: All simulations were conducted with $k = 2$ except `test_15.txt`, which used $k = 0$ (because $k = 2$ was unable to be solved in reasonable time). All simulations except `test_15.txt` obtained a lower makespan and average cost. `test_15.txt` obtained the same results as the original path.

`test_0.txt`

- Original makespan: 37.7s
- Original average cost: 20.8s
- New makespan: 25.9s
- New average cost: 17.6s

`test_1.txt`

- Original makespan: 18.9s
- Original average cost: 10.1s
- New makespan: 15.3s
- New average cost: 9.6s

`test_2.txt`

- Original makespan: 21.7s
- Original average cost: 14.1s
- New makespan: 21.7s
- New average cost: 13.4s

`test_15.txt`

- Original makespan: 36.4s
- Original average cost: 21.3s
- New makespan: 36.4s
- New average cost: 21.3s

Task 17

The first improvement was to implement symmetric constraint kR-CBS (which uses the solver `--solver KRCBS-RANGE`). This is to drastically shrink the number of nodes in the open list so that problems like `test_0.txt` with $k = 2$ can be solved. The code for this is depicted below. Essentially, instead of a single point constraint, the splitting branches two symmetric range constraints covering a set of point constraints.

```
def symmetric_range_splitting(collision, k, goals=None, t_max=None):

    a1 = collision['a1']
    a2 = collision['a2']
    loc = collision['loc']

    t = collision['timestep']
    t1 = collision['timestep_a1']
    t2 = collision['timestep_a2']

    if len(loc) == 1:
        upper1 = t2 + k
        upper2 = t1 + k

        # Constrain goal locations for up to t_max
        if goals is not None and t_max is not None:
            if loc[0] == goals[a2]:
                upper1 = t_max
            if loc[0] == goals[a1]:
                upper2 = t_max

        # Constrain a1 at loc for [max(0, t2-k) to upper1]
        branch1 = [{'agent': a1, 'loc': [loc[0]], 'timestep': t}
                   for t in range(max(0, t2 - k), upper1 + 1)]

        # Constrain a2 at loc for [max(0, t1-k) to upper2]
        branch2 = [{'agent': a2, 'loc': [loc[0]], 'timestep': t}
                   for t in range(max(0, t1 - k), upper2 + 1)]
        return [branch1, branch2]
```

```
    # Edge collision (should only occur for k=0)
    if len(loc) == 2:
        return [
            [{'agent': a1, 'loc': [loc[0], loc[1]], 'timestep': t}],
            [{'agent': a2, 'loc': [loc[1], loc[0]], 'timestep': t}],
        ]

    raise ValueError(f"Invalid collision: {collision}")
```

The second improvement is to make changes to the cost of each node in the open list to prioritize paths with less start/stops, turns, and congestion. This is done by implementing a new `get_sum_of_cost_smart` function. After some fine-tuning, the penalties are:

- Each turn was penalized with a cost of 2.0
- Each start/stop was penalized with a cost of 0.05
- Congestion (more than 2 robots in a Manhattan-radius of 1) was penalized with a cost of 0.2

The code for this is depicted below.

```
def get_direction(loc_from, loc_to):
    return (loc_to[0] - loc_from[0], loc_to[1] - loc_from[1])


def get_sum_of_cost_smart(paths, w_turn=2.0, w_congestion=0.2, w_startstop=0.05,
                          congestion_radius=1):
    if paths is None:
        return -1

    num_agents = len(paths)
    max_t = max(len(p) for p in paths)
    total = 0.0

    for i, path in enumerate(paths):

        # Move cost
        total += len(path) - 1

        for t in range(1, len(path)):
            d_curr = get_direction(path[t - 1], path[t])
            is_moving = d_curr != (0, 0)

            # Turn cost
            if t >= 2:
                d_prev = get_direction(path[t - 2], path[t - 1])
```

```
            was_moving = d_prev != (0, 0)
            if was_moving and is_moving and d_prev != d_curr:
                total += w_turn

        # Start/stop cost
        if t >= 2:
            d_prev = get_direction(path[t - 2], path[t - 1])
            was_moving = d_prev != (0, 0)
            if was_moving != is_moving:
                total += w_startstop

    # Congestion cost
    for t in range(max_t):
        locs = [get_location(paths[i], t) for i in range(num_agents)]
        for i in range(num_agents):
            for j in range(i + 1, num_agents):
                dist = abs(locs[i][0] - locs[j][0]) + abs(locs[i][1] - locs[j][1])
                if dist <= congestion_radius:
                    total += w_congestion

    return total
```

Essentially, instead of calculating the cost as the total number of steps moved for all paths, we additionally impose costs on turns, start/stops, and congestion. With these penalties, the front of the open list would hopefully be populated by paths with less start/stops, turns, and congestion. This allows the robot to behave more monotone under real robot dynamics and empirically gives lower makespan and average costs in the SMART simulator.